



TradeSense AI

System workflow reference

Version: commit fb6c01c1 | 26 Sep 2026 | Paper trading mode

What this document covers

- What TradeSense AI is and how its parts fit together
- The full daily timeline, from pre-market to end-of-day learning
- Every stage of a trading cycle, including the 5-agent debate and the decision math
- Risk limits and safety controls, and how the system learns from outcomes
- Market data, the web dashboard, Telegram alerts and commands
- How it is deployed and operated, with a runbook for common tasks

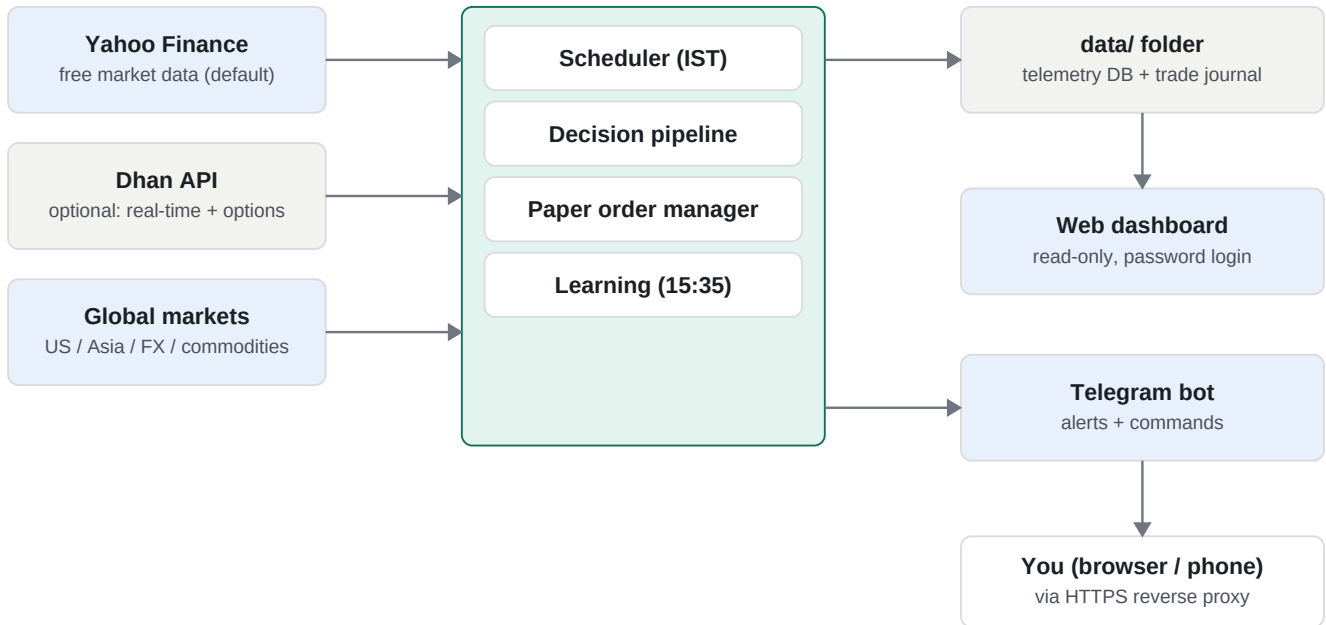
The system trades in **paper mode**: it simulates orders and records results, but never sends real orders. Nothing in this document is investment advice.

Contents

1. System overview	3
Main building blocks	3
2. Daily timeline (IST)	4
3. One trading cycle, stage by stage	5
Stage details	5
Who sets what	6
4. The five agents and the decision	7
How the Decision Engine decides	7
Worked example	7
5. Risk limits and safety controls	8
Trade lifecycle	8
6. How the system learns	9
Research modules that are not yet connected	9
How to judge performance	9
7. Market data	10
Fallback order	10
Limitations of free data	10
8. Web dashboard	11
Login and security	11
9. Telegram alerts and commands	12
Alerts	12
Who can run what	12
10. Deployment	13
Isolation rules for a shared server	13
Update to the latest main	13
Remove completely (undo)	13
Development workflow	13
11. Settings reference	14
12. Operations runbook	15
Troubleshooting	15

1. System overview

TradeSense AI is an automated trading system for Indian equities (NSE). Several times each trading day it scans the market, puts every trade idea through a chain of filters and a vote by five specialist scoring agents, and records the approved trades as **paper trades**. Every evening it learns from how those trades turned out and adjusts the next day's behaviour.



Main building blocks

Component	What it does	Where in the code
Scheduler / orchestrator	Runs every job at its IST time slot; the single entry point for a cycle	orchestrator/master_orchestrator.py
Market intelligence	Indices, VIX, breadth, PCR, sector flows, market regime	market_intelligence/, global_intelligence/
Opportunity engine	Scans stocks and builds trade signals (entry, stop, target)	opportunity_engine/
StrategyLab	Assigns a strategy, applies backtest and health gates	strategy_lab/
Risk control	Capital allocation, risk rules, stress tests, correlation, exposure	risk_control/
Simulation	Monte Carlo stability check	market_simulation/
RiskGuardian	Portfolio kill switch	risk_guardian/
Debate + Decision	5 scoring agents and the weighted decision	debate_system/, decision_ai/
Execution (paper)	Order timing, paper journal, fill records	execution_engine/
Monitoring + learning	Open-position checks, EOD learning, strategy tracking	trade_monitoring/, learning_system/, meta_learning/
Web dashboard	Read-only UI + API with password login	web_dashboard/
Telegram	Alerts to a group, operator commands	notifications/

2. Daily timeline (IST)

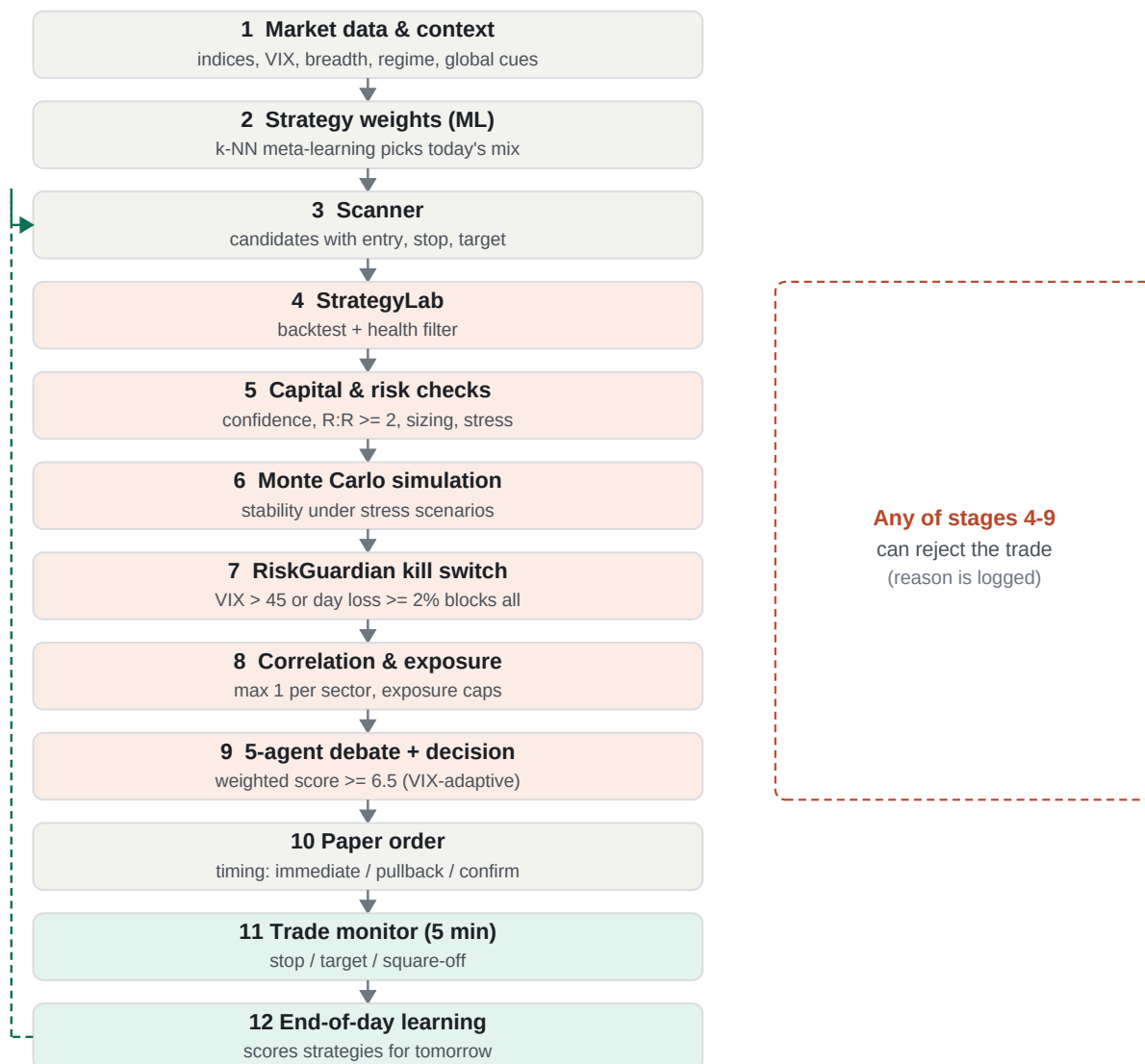
All times are India Standard Time. The engine runs continuously; the scheduler fires each job at its slot. No action is needed from you on a normal day.

Time	Job	What happens
08:00	Pre-market start	Loads data and state, runs start-up health checks
08:30 - 09:10	Warm-up	Pre-fetches global markets; pre-market refiner updates the watchlist (08:45)
09:15	Market opens	Open notification and data-feed readiness probe. No trading yet
09:45	First trading cycle	Full 12-stage decision pipeline (section 3). Trading is blocked before 09:45 to avoid the opening volatility
10:30, 11:30, 13:00, 14:00, 15:00	Trading cycles	Same pipeline; each cycle takes roughly 20-40 s
Every 5 min	Trade monitor	Checks open positions against stop-loss and target
15:05	Intraday square-off	Closes intraday (short) positions before the close
15:30	Market close	Close notification
15:35	End-of-day learning	Scores strategies, updates models (section 6). Retries at 16:00 and 17:10 if needed; alert at 17:50 if still incomplete
16:45	Post-market scan	Scans about 500 stocks (about 20 min) to prepare the next day's universe
Saturday 08:00	Weekend intelligence	Deeper accumulation and study cycle
Sunday 09:00	Monday preparation	Tactical preparation for the week

The exact slot times live in SCHEDULE in `config.py`; the dashboard's schedule panel reads them from there.

3. One trading cycle, stage by stage

A trading cycle takes the day's candidate stocks and narrows them down. Most stages can only **reject** a trade or **reduce its size**; the price levels (entry, stop, target) are set once by the scanner and never changed downstream. Approved trades become paper orders; the trade monitor and end-of-day learning close the loop.



Stage details

#	Stage	What it does	Can reject?
1	Market data & context	Pre-checks (execution window, kill switch, stale orders). Fetches indices, VIX, breadth, PCR, events and global cues; validates data integrity (aborts on hard errors); classifies the regime (bull trend, range, bear, volatile).	Aborts cycle on bad data
2	Strategy weights	A k-nearest-neighbour meta-learning model (trained on the system's own results) blends with regime rules to weight strategies for today.	No
3	Scanner	Scores the prepared universe using technical and fundamental factors. Sets entry, stop-loss and target (ATR-based), direction, confidence and reward:risk. A knowledge layer records its own evidence-based view of each signal alongside.	No (creates signals)
4	StrategyLab	Assigns the best-fit strategy, applies evolved parameters, and filters by backtest quality and strategy health (strategies that keep losing are disabled).	Yes

#	Stage	What it does	Can reject?
5	Capital & risk checks	Caps the number of new positions; RiskManager requires confidence ≥ 6.0 , reward:risk ≥ 2.0 , a defined stop and available risk budget; position sizing; stress tests. Rejections are logged for later analysis.	Yes
6	Monte Carlo simulation	Simulates each trade across stress scenarios and rejects unstable ones.	Yes
7	RiskGuardian	Hard kill switch at portfolio level: blocks everything if VIX > 45 or the day's loss reaches 2% of capital. Cannot be bypassed.	Yes (blocks all)
8	Correlation & exposure	At most one position per sector; exposure caps and VIX-adjusted allocation.	Yes
9	Debate + decision	Five scoring agents vote; the Decision Engine combines them into a weighted score and decides full, half or no size (section 4). A data-truth governor then blocks trades built on synthetic prices.	Yes
10	Paper order	Order manager chooses timing (immediate, wait for pullback, or wait for confirmation), re-checks the kill switch before any deferred placement, and writes the trade to the paper journal.	Kill switch re-check
11	Trade monitor	Every 5 minutes: exits at stop-loss or target, applies square-off rules.	-
12	End-of-day learning	Updates strategy statistics and models (section 6).	-

Who sets what

Parameter	Set by
Entry, stop-loss, target, direction, confidence, reward:risk	Scanner (stage 3)
Strategy name	StrategyLab (stage 4)
Position size	Capital/risk sizing (stage 5), exposure engine (stage 8), decision modifier (stage 9)
Final go / no-go	Decision Engine (stage 9), then data-truth governor and order manager

4. The five agents and the decision

The agents are **rule-based scoring programs**, not calls to an external AI service. Each looks at the trade from one angle and returns a score from 0 to 10, a vote (approve, reduce size, hedge, reject), a suggested size modifier and a written reason. No AI API key is required and there is no per-use cost. Code:

`debate_system/multi_agent_debate.py`.

Agent	Weight	Question it answers	Key rules
Technical Analyst	30%	Is the trade structurally sound?	Buy: stop < entry < target (inverse for sells), else reject (3.0). Stop distance must be 0.5-4x the stock's ATR, else weak (5.0, size 70%). Sound: 8.0.
Risk	25%	Is volatility acceptable and the payoff worth it?	VIX < 18: base 7.0; 18-22: 6.0 at 75% size; 22+: 4.5 at 50% size. Reward:risk bonus: >= 4: +1.5, >= 3: +1.0, >= 2: +0.5, < 1.5: -1.5 (and 50% size).
Macro	20%	What are global markets saying?	Event day (RBI, results, expiry): hedge, 5.0 at 60% size. Bear market: reject (3.0). Otherwise 7.0 adjusted by overnight global sentiment, range 5-9.
Sentiment	15%	How is the crowd positioned?	For buys: PCR > 1.2 is contrarian bullish (7.0); breadth < 35% means risk-off (5.5 at 60% size); otherwise neutral 7.0.
Regime	10%	Does the strategy suit today's market?	Strategy matches the regime's allowed list: 8.0; mismatch: 5.0 at 70% size. Signals already approved by the evidence-based knowledge layer score 8.0.

How the Decision Engine decides

- **Weighted score** = sum of (agent score x agent weight). The weights add up to 1.
- **Size modifier** = geometric mean of the agents' suggested modifiers.
- **Threshold** depends on VIX: 6.5 (VIX < 20), 6.6 (20-25), 6.7 (25-30), 6.9 (30+). High reward:risk lowers it: -0.5 for R:R >= 3, -1.0 (and +10% size) for R:R >= 4.
- **Outcome**: score >= threshold: approve at full size. Within 0.2 below: approve at **half** size. Otherwise reject.

Note: `config.py` also defines `MIN_CONFIDENCE_SCORE` = 6.8, but the Decision Engine only prints it at start-up; the gate actually applied is the VIX-adaptive threshold above. This mismatch is worth tidying up in the code.

Worked example

Buy SBIN; VIX 12; bull trend; reward:risk 2.0; a momentum strategy; neutral sentiment; mildly positive global cues (+0.2).

Agent	Score	x Weight	Contribution
Technical	8.0	0.30	2.40
Risk	7.5 (7.0 + 0.5)	0.25	1.88
Macro	7.4 (7.0 + 0.2 x 2)	0.20	1.48
Sentiment	7.0	0.15	1.05
Regime	8.0	0.10	0.80
Weighted score	7.6 >= 6.5: APPROVED, full size		

With VIX at 20, reward:risk 1.4 and a strategy that does not suit the regime, the same trade scores about 6.5 against a raised threshold and is rejected.

5. Risk limits and safety controls

Values below are the code defaults. Position sizes scale with TOTAL_CAPITAL (set per installation; the current deployment uses Rs 2,00,000).

Control	Limit	Effect
Paper trading	PAPER_TRADING=true (default)	Orders are simulated. Live orders additionally need PAPER_TRADING=false and LIVE_TRADING_AUTHORIZED=true; otherwise the system forces paper mode.
Risk per trade	0.25% of capital	Quantity = risk amount / stop distance (e.g. Rs 500 on Rs 2 lakh)
Daily loss kill switch	2% of capital	RiskGuardian halts all new trades for the day
Volatility kill switch	India VIX > 45	RiskGuardian blocks all new trades
Risk at stake	5% of capital (RiskGuardian)	Caps combined open risk
Max positions	8 for capital above Rs 1 lakh (3 / 5 for smaller)	Override with MAX_POSITIONS
Drawdown halt	10%	Trading halts on a 10% drawdown
Sector correlation	1 position per sector	Avoids concentrated bets
Minimum quality	Confidence >= 6.0, reward:risk >= 2.0	RiskManager rejects weaker trades
Execution window	No trades before 09:45	Avoids the opening volatility
Data truth	Synthetic equity prices	Trade approval is suppressed if prices are not real

Trade lifecycle

- **Entry:** buys use delivery (CNC) so they can be held for the strategy's planned 3-7 day horizon; shorts are intraday, as SEBI requires for uncovered cash-market shorts.
- **While open:** the trade monitor checks stop-loss and target every 5 minutes.
- **Exit:** stop hit, target hit, intraday square-off at 15:05 (shorts), or end of the holding period.
- **Record:** every open and close is written to data/paper_trades.csv with entry, exit, P&L and reason.

6. How the system learns

Learning uses only the system's own trade outcomes, processed on the server. No external AI service is involved.

When	Learning step	Effect on future trades
15:35 daily	Strategy performance tracker	Updates each strategy's win rate and expectancy; persistently losing strategies are disabled in StrategyLab the next day
15:35 daily	Regime-strategy map	Learns which strategies work in which market regime
15:35 daily	Meta-learning model	Retrains the k-NN strategy-weighting model when due
15:35 daily	Edge discovery	Mines outcomes for improved strategy variants; promoted variants go into the next day's strategy pool
15:35 daily	Knowledge outcomes	Fills in what each earlier signal actually did over the next 1-5 days; feeds the evidence-based knowledge layer
Each cycle	Rejection tracker	Records why trades were rejected; used by knowledge fusion
After 30+ trades	Validation engine	Statistical validation of the strategy set

Research modules that are not yet connected

The repository also contains research pipelines (autonomous research, options research, production-readiness gates) that run or can be run, but whose output does not yet change live decisions. `SYSTEM_OPERATING_MAP.md` lists exactly which pipelines feed decisions and which do not.

How to judge performance

Judge over weeks, not days. The project's own bar before considering live trading: win rate $\geq 50\%$, Sharpe ratio > 0.8 and maximum drawdown $< 15\%$. Few or no trades in the first days is normal while the filters have no history.

7. Market data

Without a broker connection the system uses **Yahoo Finance** through the open-source `yfinance` library: no account, no key, no cost.

Data	Yahoo symbol	Refresh
NSE stocks	SYMBOL.NS (e.g. SBIN.NS)	Prices cached 60 s; history on demand
NIFTY 50 / Bank Nifty	^NSEI / ^NSEBANK	Each cycle
India VIX	^INDIAVIX	Each cycle
Global markets	^GSPC, ^N225, USDINR=X, GC=F, CL=F ...	Every 5 min in the background

Fallback order

Dhan (if a token is configured) > Yahoo Finance > last good cached price > built-in estimate. If equity prices become synthetic, the data-truth governor **blocks new trades** rather than trading on bad data.

Limitations of free data

- Yahoo NSE quotes are delayed (typically around 15 minutes), so paper fills are approximate.
- No NSE option chain: options strategies stay switched off; only stock trades are taken.
- No market depth, so slippage cannot be modelled precisely.
- Unofficial source: occasional throttling shows up as degraded feed status on the dashboard.

Adding a Dhan account and API token later gives real-time prices, the option chain and exact tick sizes, with no code changes.

8. Web dashboard

A read-only web app (FastAPI backend + React frontend) that reads the engine's telemetry database and trade journal. It cannot place, change or cancel trades. It refreshes every 5 seconds and works on phones.

Page	What you see
Overview	Total and today's P&L, win rate, open positions, account equity, equity curve, latest cycle (regime, VIX, PCR, breadth, signal funnel), today's schedule, recent decisions
Trades & P&L	Profit factor, average win/loss, max drawdown, daily P&L, P&L by strategy, exit reasons, searchable closed-trades table
Signals & decisions	Where signals drop out, top rejection reasons, decisions by strategy, decision log with each agent's score
System health	Engine status, cycle durations, agent health, live event stream, end-of-day report

Login and security

- Password stored only as a salted PBKDF2-SHA256 hash (DASHBOARD_PASSWORD_HASH); set it with `python -m web_dashboard.set_password`.
- Session: signed HttpOnly, SameSite=Strict cookie; expires after 12 hours; changing the password signs everyone out.
- Fails closed: with no password configured, no data can be read.
- Brute-force protection: 5 wrong attempts lock that visitor out for 15 minutes. Behind a reverse proxy the real visitor IP is taken from X-Forwarded-For only for proxies listed in DASHBOARD_TRUSTED_PROXIES.

9. Telegram alerts and commands

The engine posts alerts to a Telegram group and answers commands. In a group, add @<bot username> after a command (for example /status@TradeSenseAI_bot); in a private chat it is not needed.

Alerts

- Engine start-up and online status (plus a Dhan token prompt while no broker is connected)
- Trade approvals and exits with entry, stop, target and P&L
- Kill-switch and data-feed warnings
- End-of-day summary after 15:35

Who can run what

Sender	Read-only commands	Control: /pause /resume /rescan	/token
Any member of the registered group	Yes	No (restricted to administrators)	No
Admin (listed in TELEGRAM_WHITELIST_IDS), in the group	Yes	Yes	No (send it privately)
Admin, private chat with the bot	Yes	Yes	Yes
Anyone else, private chat	No (Unauthorized)	No	No

Useful read-only commands: /status, /pnl, /positions, /cycle, /perf, /market, /learn, /report, /eod, /help. Never post a broker token in a group; the bot refuses /token there.

10. Deployment

The system runs as two Docker containers on a Linux VPS, alongside other applications that must not be disturbed. A shared Caddy reverse proxy owns ports 80/443 and serves HTTPS; each application is added as one site block.

Item	Setup
Folder	/opt/tradesense (a clone of the main branch)
Docker project	tradesense (COMPOSE_PROJECT_NAME in .env keeps every compose command scoped)
Containers	ai-trading-brain (engine, capped at 3 GB / 1 CPU) and trading-dashboard (1 GB / 0.5 CPU)
Networking	Dashboard has no public port; it joins the shared proxy network via a server-only docker-compose.override.yml
HTTPS	Reverse proxy site block for the dashboard domain with an automatic Let's Encrypt certificate; DNS record is DNS-only (not proxied)
Settings	.env on the server only (never committed): paper mode, capital, dashboard hash, Telegram
Image build	Multi-stage Dockerfile: Node builds the React frontend; the Python image serves it

Isolation rules for a shared server

- Run compose commands only inside /opt/tradesense; never docker system prune, and never touch other projects' containers, files or databases.
- Change the reverse proxy only by: back up the Caddyfile, append the site block in place, validate, then reload (graceful, no downtime). Never restart the shared proxy container.
- Snapshot the list of running containers before and after any change and compare.

Update to the latest main

```
cd /opt/tradesense && git pull && docker compose build && docker compose up -d
```

Remove completely (undo)

```
cd /opt/tradesense && docker compose down
# restore the Caddyfile backup, validate and reload the proxy
rm -rf /opt/tradesense
```

Development workflow

- Changes are made on a feature branch, tested (pytest + frontend build), and merged to main by pull request; the server then pulls main.
- Frontend: `cd web_dashboard/frontend && pnpm install && pnpm build`.
- Local run (Windows, no Docker): set `RUNNING_IN_DOCKER=1` and run `python main.py --paper`; dashboard: `python -m web_dashboard.server`.

11. Settings reference

All settings are environment variables in `.env`, which is git-ignored. `.env.example` in the repository documents them without real values.

Setting	Purpose
PAPER_TRADING / LIVE_TRADING_AUTHORIZED	Paper mode unless BOTH are set for live trading
TOTAL_CAPITAL	Capital used for sizing (rupees)
MAX_POSITIONS	Optional override of the max simultaneous positions
ACTIVE_BROKER, DHAN_CLIENT_ID, DHAN_ACCESS_TOKEN	Broker selection and Dhan credentials (optional)
TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID	Bot token and the chat (or group) that receives alerts
TELEGRAM_WHITELIST_IDS	Telegram user ids allowed to run control commands
DASHBOARD_PASSWORD_HASH	Dashboard login (hash only)
DASHBOARD_SESSION_HOURS	Login lifetime (default 12)
DASHBOARD_COOKIE_SECURE	true when served over HTTPS
DASHBOARD_TRUSTED_PROXIES	Reverse-proxy IPs/CIDRs trusted for the real visitor IP
DASHBOARD_PORT / DASHBOARD_HOST	Dashboard listen address (default 0.0.0.0:8501)
COMPOSE_PROJECT_NAME	Docker project name on the server (tradesense)

12. Operations runbook

Task	How
Daily check	Dashboard header shows Engine online and Market open from 09:45; Telegram posts appear; System health shows cycles completing
Set / change dashboard password	On the server: <code>docker compose exec ai-trading-brain python -m web_dashboard.set_password</code> , then restart the dashboard container
View engine logs	<code>docker logs --tail 200 ai-trading-brain</code>
Restart only TradeSense	<code>cd /opt/tradesense && docker compose restart</code>
Pause / resume trading	Telegram (admin): <code>/pause</code> or <code>/resume</code>
Connect Dhan data	Add <code>DHAN_CLIENT_ID</code> and <code>DHAN_ACCESS_TOKEN</code> to the server <code>.env</code> and restart the engine, or send <code>/token</code> to the bot in a private chat
Change capital	Edit <code>TOTAL_CAPITAL</code> in the server <code>.env</code> and restart the engine
Server reboot	Only at a quiet time (it restarts every application on the server); all containers restart automatically

Troubleshooting

Symptom	Likely cause and fix
Dashboard shows 'No dashboard password is set'	<code>DASHBOARD_PASSWORD_HASH</code> missing: run <code>set_password</code> and restart the dashboard
Engine offline / idle in the header	No telemetry for 10+ min: check <code>docker compose ps</code> and the engine logs
No trades for days	Normal early on; check Signals & decisions for the stage rejecting most signals
Feed degraded / <code>YAHOO_FALLBACK</code>	Yahoo throttling or outage; system falls back and blocks trades on synthetic prices
Telegram silent	Check the engine log for 'Telegram=enabled'; a basic group converted to a supergroup gets a new -100... id, so update <code>TELEGRAM_CHAT_ID</code>
Locked out of the dashboard	5 failed attempts: wait 15 minutes